

Práctica 6. Reconocimiento de voz y de la mano con IA

Ejercicio 1. Funcionamiento del sistema

En esta práctica programarás dos scripts de Python que agregan inteligencia artificial a lo que capta el ESP32. El primero, `voz_a_texto.py`, recibe el audio del micrófono y reconoce las palabras que se dicen, mostrándolas como texto en la terminal. El segundo, `senas_mano.py`, recibe el video de la cámara y reconoce los puntos de la mano en cada imagen. Cada script parte de una copia de la plantilla que ya usaste en clase para recibir el audio o el video (Tabla 1). El ESP32 solo captura y envía por UDP con los programas del proyecto `clase_audio_video`, sin modificarlos más que en la red WiFi (ver la Nota); todo el reconocimiento lo hace la PC en python.

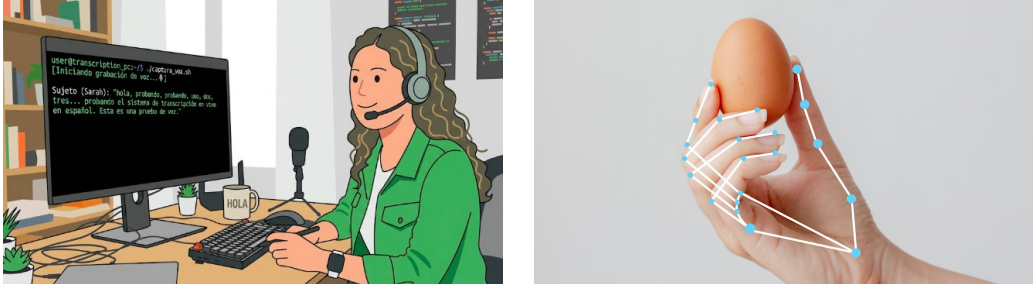


Fig 1. Voz a texto y reconocimiento de puntos de la mano.

Prueba los programas como están, programa en la ESP32 el envío del audio, y prueba el programa en Python en la computadora. Después, prueba el envío de video, y el programa en Python correspondiente.

Tabla 1. Programas de la práctica

Programa del ESP32 (en <code>main.c</code>)	Plantilla → script a programar	Librería
Audio: 1_Microfono_UDP	<code>graficar_audio.py</code> → <code>voz_a_texto.py</code>	faster-whisper
Video: 2_Camara_UDP	<code>ver_video.py</code> → <code>senas_mano.py</code>	mediapipe

Nota. El programa del ESP32 trabaja en **modo estación (STA)**: se une a una red WiFi que ya existe. Antes de compilar, cambia `WIFI_SSID` y `WIFI_PASS` en `1_Microfono_UDP.c` y `2_Camara_UDP.c` por los de la red que vayas a usar (de 2.4 GHz). La PC debe estar conectada a la misma red, y en `IP_ESP32` del script va la IP que imprime el monitor serie.

Ejercicio 2. Voz a texto (`voz_a_texto.py`)

- Recibe el audio igual que en `graficar_audio.py`.
- Junta algunos segundos de audio y transcríbelos al español con **faster-whisper**.
- Imprime en la terminal el texto de lo que se dice al micrófono del ESP32.

Ejercicio 3. Detección de la mano (`senas_mano.py`)

- Recibe las fotos igual que en `ver_video.py`.
- Detecta la mano con **MediaPipe Hand Landmarker**.
- En la ventana del video, dibuja los 21 puntos de la mano y una línea entre la punta del pulgar y la del índice, y escribe la distancia entre ellas.

Manual 1. faster-whisper (voz a texto)

Fuente oficial (secciones Installation, Usage y VAD filter):
<https://github.com/SYSTRAN/faster-whisper>

Instalación (con tu ambiente `venv_embebidos` activado)

```
pip install faster-whisper
```

Uso

El código **no es un programa completo**: son fragmentos que se agregan a una copia de `graficar_audio.py` llamada `voz_a_texto.py`. Cada encabezado `# ===` indica dónde va el fragmento.

```
# === 1) AL INICIO, junto a los demas import ===
from faster_whisper import WhisperModel

# === 2) ANTES del ciclo while True: cargar el modelo una sola vez ===
# Tarda unos segundos y la primera vez lo descarga de internet.
# Tamano: "tiny", "base" o "small" (mas grande: mejor, pero mas lento)
modelo = WhisperModel("base", device="cpu", compute_type="int8")

# === 3) DENTRO del ciclo, despues de recibir datos ===
# TU PARTE: cada datagrama (datos) trae 32 ms de audio. Juntalos en la
# variable pendiente hasta tener unos segundos:
# 3 s = 3 x 16000 muestras/s x 2 bytes = 96000 bytes.
# Cuando pendiente ya tenga esos bytes:

# 3a) El ESP32 manda enteros de 16 bits; Whisper pide decimales de -1 a 1
audio = np.frombuffer(pendiente, dtype=np.int16).astype(np.float32)
audio = audio / 32768

# 3b) Transcribir en espanol; vad_filter=True salta el silencio
segmentos, info = modelo.transcribe(audio, language="es",
                                     vad_filter=True)

for segmento in segmentos:
    print(segmento.text)

pendiente = b"" # vaciar para juntar los siguientes segundos
```

Manual 2. MediaPipe Hand Landmarker (la mano)

Instalación (con tu ambiente `venv_embebidos` activado)

```
pip install mediapipe
```

Descarga el modelo de la mano `hand_landmarker.task` y guárdalo en `practicas/2_Camara_UDP/`, junto a tu script. El enlace está en la sección Models de la guía oficial, que también muestra los 21 puntos de la mano:
https://developers.google.com/edge/mediapipe/solutions/vision/hand_landmarker

https://storage.googleapis.com/mediapipe-models/hand_landmarker/hand_landmarker/float16/latest/hand_landmarker.task

Uso

El código **no es un programa completo**: son fragmentos que se agregan a una copia de `ver_video.py` llamada `senas_mano.py`. Cada encabezado `# ===` indica dónde va el fragmento. Se usa la API **Tasks**: los ejemplos de internet con `mp.solutions` ya no funcionan en las versiones nuevas de MediaPipe. Fuente oficial (guía para Python):
https://developers.google.com/edge/mediapipe/solutions/vision/hand_landmarker/python

```

# === 1) AL INICIO, junto a los demas import ===
from pathlib import Path
import mediapipe as mp
from mediapipe.tasks.python import vision, BaseOptions

# === 2) ANTES del ciclo while True: crear el detector una sola vez ===
# El modelo hand_landmarker.task esta en la carpeta del script
ruta = Path(__file__).parent / "hand_landmarker.task"
opciones = vision.HandLandmarkerOptions(
    base_options=BaseOptions(model_asset_path=str(ruta)),
    num_hands=1) # buscar una sola mano
detector = vision.HandLandmarker.create_from_options(opciones)

# === 3) DENTRO del ciclo, despues de obtener foto y antes de imshow ===
# 3a) MediaPipe pide colores RGB y OpenCV los da en BGR
rgb = cv2.cvtColor(foto, cv2.COLOR_BGR2RGB)
imagen = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
resultado = detector.detect(imagen)

# 3b) Una entrada por mano encontrada; cada mano tiene 21 puntos (0 a 20)
alto, ancho, _ = foto.shape
for mano in resultado.hand_landmarks:
    pulgar = mano[4] # 4: punta del pulgar
    indice = mano[8] # 8: punta del indice
    # x y y van de 0 a 1: a pixeles, por el ancho y el alto de la foto
    x = int(pulgar.x * ancho)
    y = int(pulgar.y * alto)

    # Dibuja sobre foto los 21 puntos y sus uniones
    vision.drawing_utils.draw_landmarks(
        foto, mano, vision.HandLandmarksConnections.HAND_CONNECTIONS)

    # TU PARTE: linea entre pulgar e indice (cv2.line) y la distancia
    # entre ellos escrita en la foto (cv2.putText)

```

Reporte

Elabora un reporte siguiendo el formato de la plantilla Reporte_plantilla.docx proporcionada por el profesor. Incluye capturas de la terminal con el texto transcrito y de la ventana del video con la mano detectada. Los siguientes puntos te pueden ayudar a redactar tu reporte.

1. Objetivos. Escribe los objetivos que identificaste en la práctica. Para ayudarte, puedes identificar el tema abordado y el aprendizaje obtenido.
2. Resultados y análisis. El análisis puede estar basado en las siguientes preguntas:
 - a. ¿Qué modelo de Whisper usaste (tiny, base o small)? Compara cuánto tarda y qué tan bien transcribe cada uno.
 - b. ¿Cuántos segundos de audio juntas antes de transcribir? ¿Qué pasa si una palabra queda partida entre dos bloques?
 - e. ¿En qué condiciones falla la detección de la mano (luz, distancia, fondo, varias manos)?
3. Conclusiones. Puedes redactar tus conclusiones basado en las siguientes preguntas:
 - a. ¿Por qué el reconocimiento lo hace la PC y no el ESP32? ¿Qué se necesitaría para hacerlo en el ESP32?
 - b. ¿Qué ventajas y desventajas tiene usar de IA locales, como estos, en lugar de un servicio en la nube?
 - c. ¿Qué aplicaciones reales tiene un dispositivo que capta audio o video y una computadora que lo reconoce?
 - d. ¿Cómo podrías hacer más inteligente el reconocimiento de voz y la detección de puntos de la mano y para qué aplicaciones?